

目录

Kimi Code CLI 使用指南（团队版）

一、Kimi Code CLI 是什么

二、环境要求

三、安装步骤

3.1 一键安装（推荐）

3.2 如果你已经有 uv

3.3 验证安装

3.4 安装后命令找不到？

四、首次登录与配置

4.1 启动

4.2 API平台

4.3 登录

4.4 远程服务器 / 无图形界面环境

五、快速上手三步走

第一步：问一个问题

第二步：做一次代码修改

第三步：执行一条命令

初始化项目描述文件

六、常用命令速查

6.1 启动命令

6.2 交互中斜杠命令

6.3 快捷键

七、核心交互技巧

7.1 @ 路径补全

7.2 审批与确认

7.3 运行中引导 (Steer)

7.4 侧问

八、工作模式详解

8.1 Agent 模式 vs Shell 模式

8.2 Plan 模式 (先规划再动手)

8.3 Thinking 模式 (深度思考)

8.4 YOLO 模式 (自动批准)

8.5 Print 模式 (脚本/CI 集成)

九、会话管理

9.1 继续之前的会话

9.2 运行中切换会话

9.3 会话状态自动持久化

9.4 导出/导入

十、上下文管理

十一、高级配置

11.1 配置文件位置

11.2 手动配置其他 LLM 平台

11.3 搜索和抓取服务

十二、升级与卸载

十三、常见问题 FAQ

Kimi Code CLI 使用指南（团队版）

本文档基于 Kimi Code 官方文档整理，供组内成员快速上手使用。

官方文档地址：<https://www.kimi.com/code/docs/kimi-code-cli/getting-started.html>

一、Kimi Code CLI 是什么

Kimi Code CLI 是一个运行在终端中的 AI Agent 编程助手（类似 Claude Code），能够帮你完成以下工作：

- **编写和修改代码：**实现新功能、修复 bug、重构代码
- **理解项目：**探索陌生代码库，解答架构和实现问题
- **自动化任务：**批量处理文件、执行构建和测试、运行脚本
- **搜索和抓取网页：**联网搜索资料（仅 Kimi Code 平台支持）

它有三种使用方式：

方式	命令	适用场景
交互式命令行	<code>kimi</code>	日常开发，在终端中与 AI 对话
浏览器界面	<code>kimi web</code>	偏好图形界面，支持会话管理、代码高亮
Agent 集成	<code>kimi acp</code>	集成到 IDE 或其他 Agent 客户端

二、环境要求

- **操作系统：** macOS / Linux / Windows（通过 PowerShell）
 - **账号：** 需要 Kimi 会员订阅，或可调用的 API Key
 - **Python：** 安装脚本会自动安装 uv（Python 包管理工具），支持 Python 3.12~3.14，推荐 3.13
-

三、安装步骤

3.1 一键安装（推荐）

安装脚本会自动安装 uv + Kimi Code CLI：

```
# Linux / macOS
curl -LsSf https://code.kimi.com/install.sh | bash

# Windows (PowerShell)
Invoke-RestMethod https://code.kimi.com/install.ps1 | Invoke-Expression
```

3.2 如果你已经有 uv

```
uv tool install --python 3.13 kimi-cli
```

3.3 验证安装

```
kimi --version
```

3.4 安装后命令找不到？

安装脚本会将 `kimi` 添加到 PATH，但需要**重启终端**或执行以下命令才能生效：

```
source ~/.bashrc    # bash 用户
source ~/.zshrc     # zsh 用户
```

如果仍然找不到，检查 `~/local/bin` 是否在你的 PATH 中。

四、首次登录与配置

4.1 启动

在你的项目目录中启动：

```
cd your-project #cd到目标项目目录  
kimi
```

4.2 API平台

```
https://platform.kimi.com/console/account
```

4.3 登录

首次启动后，输入 `/login` 配置 API 来源：

```
/login
```

会出现平台选择界面，推荐选择 **API**，接着提示输入API KEY。输入4.2中生成的API KEY即可。

4.4 远程服务器 / 无图形界面环境

如果浏览器没有弹出，`/login` 会显示一个 URL，手动复制到浏览器打开即可完成授权。

五、快速上手三步走

第一步：问一个问题

用自然语言提问，快速了解项目：

这个项目的整体架构是怎样的？入口文件在哪里？

Kimi Code CLI 会自动搜索和阅读相关文件，然后给出回答。

第二步：做一次代码修改

给 README 添加一个"快速开始"部分，包含安装和运行步骤

修改文件前会展示 diff 并请求确认——你可以批准、拒绝，或输入反馈让它调整。
不会未经允许改动代码。

第三步：执行一条命令

运行测试，如果有失败的用例就修复它们

至此你已体验三个核心能力：提问理解 → 修改代码 → 执行命令。

初始化项目描述文件

如果项目中没有 `AGENTS.md` 文件，建议运行：

```
/init
```

让 Kimi Code CLI 分析项目并生成该文件，帮助 AI 更好地理解项目结构和规范。

六、常用命令速查

6.1 启动命令

命令	说明
<code>kimi</code>	启动交互式对话
<code>kimi web</code>	打开浏览器图形界面
<code>kimi --continue</code>	继续当前目录下最近的会话
<code>kimi --session</code>	交互式选择历史会话
<code>kimi -r <session-id></code>	恢复指定会话
<code>kimi --plan</code>	以 Plan（规划）模式启动
<code>kimi --thinking</code>	启用 Thinking（深度思考）模式
<code>kimi --yolo</code>	启用 YOLO 模式（自动批准所有操作）
<code>kimi --print -p "指令"</code>	非交互模式，执行完自动退出

6.2 交互中斜杠命令

命令	说明
<code>/login</code>	配置或切换 API 来源
<code>/usage</code>	查看剩余额度和配额

命令	说明
/model	切换模型和 Thinking 模式
/help	查看所有命令和快捷键
/init	分析项目并生成 AGENTS.md
/new	创建并切换到新会话
/sessions	查看/切换历史会话
/compact	压缩上下文（保留关键信息，减少 token）
/clear	清空当前会话上下文
/export	导出会话为 Markdown 文件
/import	从文件或其他会话导入上下文
/plan	切换 Plan 模式开关
/yolo	切换 YOLO 模式开关
/btw <问题>	侧问（不打断主对话流程）
/task	查看后台任务状态
/title <文字>	为当前会话设置标题

6.3 快捷键

快捷键	说明
Enter	发送消息
Ctrl-J / Alt-Enter	换行（不发送）
Ctrl-V	粘贴文本/图片/视频
Ctrl-X	在 Agent 模式和 Shell 模式之间切换
Ctrl-E	展开查看完整内容（diff/方案）
Ctrl-O	打开外部编辑器编辑输入
Ctrl-C	中断当前操作
Ctrl-D	退出
Ctrl-S	立即注入引导消息到当前轮次
Shift-Tab	切换 Plan 模式

七、核心交互技巧

7.1 @ 路径补全

在消息中输入 `@` 可以引用项目中的文件，会自动补全：

帮我看一下 `@src/components/Button.tsx` 这个文件有没有问题

在 Git 仓库中优先使用 `git ls-files` 查询，在大型仓库中也能快速定位。

7.2 审批与确认

AI 执行修改文件、运行命令等操作前会请求确认，你可以：

- **允许**：执行这次操作
- **本会话允许**：当前会话中自动批准同类操作
- **拒绝**：不执行
- **附带反馈拒绝**：拒绝并输入文字反馈，指导 AI 调整

7.3 运行中引导 (Steer)

AI 执行任务时，你可以直接输入消息调整方向：

- `Enter`：排队，当前轮次完成后发送
- `Ctrl-S`：立即注入当前轮次
- `Ctrl-C`：中断

7.4 侧问

AI 工作期间想问个快速问题但不打断主流程：

/btw 这个函数的返回类型是什么？

响应显示在模态面板中， `Escape` 关闭。

八、工作模式详解

8.1 Agent 模式 vs Shell 模式

- **Agent 模式**（默认）：输入发送给 AI 处理
- **Shell 模式**：直接执行 Shell 命令

按 `ctrl-x` 切换。当前模式显示在底部状态栏。

8.2 Plan 模式（先规划再动手）

适合复杂任务，让 AI 先制定方案再执行：

```
/plan on      # 开启  
/plan off     # 关闭  
/plan view    # 查看当前方案
```

Plan 模式下 AI 只能读代码、不能改代码，方案完成后提交审批，你确认后才开始执行。

8.3 Thinking 模式（深度思考）

让 AI 回答前进行更深入的推理：

```
kimi --thinking      # 启动时开启
```

或通过 `/model` 命令切换模型时选择是否开启。

8.4 YOLO 模式（自动批准）

跳过所有确认提示，适合在可控环境中使用：


```
kimi --yolo          # 启动时开启  
/yolo                # 运行中切换
```

注意：YOLO 模式会自动批准所有操作，请确保在安全环境中使用。

8.5 Print 模式（脚本/CI 集成）

非交互模式，适合自动化场景：

```
# 基本用法  
kimi --print -p "列出当前目录的所有 Python 文件"  
  
# 只输出最终结果  
kimi --quiet -p "根据当前变更给我一个 Git commit message"  
  
# 批量处理  
for file in src/*.py; do  
    kimi --print -p "为 $file 添加类型注解"  
done
```

退出码： 0 成功， 1 不可重试失败， 75 可重试失败。

九、会话管理

9.1 继续之前的会话

```
kimi --continue      # 继续最近的会话
kimi --session       # 交互式选择会话
kimi -r              # 恢复指定会话
```

9.2 运行中切换会话

```
/sessions           # 查看并切换 (Ctrl-A 切换显示范围)
/new                 # 创建新会话
```

9.3 会话状态自动持久化

恢复会话时以下状态会自动还原：审批决策、Plan 模式状态、子 Agent 实例、额外工作目录。

9.4 导出/导入

```
/export              # 导出为 Markdown
/export ~/exports/my-session.md
/import ./previous.md # 从文件导入
/import abc12345      # 从会话 ID 导入
```

十、上下文管理

底部状态栏会显示上下文使用率，如 `context: 42.0% (4.2k/10.0k)`。

命令	说明
<code>/compact</code>	压缩上下文（AI 总结后替换原内容）
<code>/compact</code> 保留数据库相关的讨论	带指引的压缩
<code>/clear</code>	清空所有上下文

十一、高级配置

11.1 配置文件位置

配置自动保存在 `~/.kimi/config.toml` 。

11.2 手动配置其他 LLM 平台

Kimi CLI 支持多种 LLM 平台，可在配置文件中手动添加：

Kimi API:

```
[providers.kimi-for-coding]
type = "kimi"
base_url = "https://api.kimi.com/coding/v1"
api_key = "sk-xxx"
```

OpenAI:

```
[providers.openai]
type = "openai_legacy"
base_url = "https://api.openai.com/v1"
api_key = "sk-xxx"
```

Anthropic Claude:

```
[providers.anthropic]
type = "anthropic"
base_url = "https://api.anthropic.com"
api_key = "sk-ant-xxx"
```

Google Gemini:

```
[providers.gemini]
type = "gemini"
base_url = "https://generativelanguage.googleapis.com"
api_key = "xxx"
```

支持的供应商类型: `kimi`、`openai_legacy`、`openai_responses`、`anthropic`、`gemini`、`vertexai`。

11.3 搜索和抓取服务

`SearchWeb` 和 `FetchURL` 工具目前仅 **Kimi Code 平台支持**。使用其他平台时，`FetchURL` 会回退到本地抓取，`SearchWeb` 不可用。

十二、升级与卸载

```
# 升级
uv tool upgrade kimi-cli --no-cache

# 卸载
uv tool uninstall kimi-cli
```

十三、常见问题 FAQ

Q: 填了 API Key 提示鉴权失败?

A: 确认 Key 和 Base URL 属于同一个平台。 `api.kimi.com` 和 `api.moonshot.cn` 的 Key 互不通用。

Q: `/login` 后浏览器没弹出?

A: 远程服务器环境下会显示一个 URL，手动复制到浏览器打开即可。

Q: 如何查看剩余额度?

A: 在交互界面中输入 `/usage`。

Q: 上下文太长怎么办?

A: 使用 `/compact` 压缩，或 `/new` 开启新会话。

Q: 如何反馈问题?

A: GitHub Issues: <https://github.com/MoonshotAI/kimi-cli/issues>

最后更新: 2026-04-25

完整官方文档: <https://www.kimi.com/code/docs/>